



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

Experiment 13: Write the python program to implement Minimax algorithm for gaming

Aim: To write and execute a Python program to implement the Minimax algorithm for game playing.

Algorithm:

1. Start the program.
2. Define the game tree with possible game states.
3. Define the Minimax function.
4. Check whether the current node is a terminal node.
5. If it is a terminal node, return its utility value.
6. If it is the MAX player's turn, select the maximum value among the child nodes.
7. If it is the MIN player's turn, select the minimum value among the child nodes.
8. Continue until the best move is found.
9. Display the optimal value.
10. Stop the program.

Program code:

```
def minimax (depth, node, max_ player, values):  
    if depth == 2:  
        return values[node]  
    if max_ player:  
        return max (  
            minimax (depth+1, node*2, False, values),  
            minimax (depth+1, node*2+1, False, values)  
        )
```

```

else:
    return min (
        minimax (depth+1, node*2, True, values),
        minimax (depth+1, node*2+1, True, values)
    )
values = [3, 5, 2, 9]
result = minimax (0, 0, True, values)
print ("Optimal Value:", result)

```

OUTPUT:

```

Tic-Tac-Toe
You are X, Computer is O

|  |
--+--+
|  |
--+--+
|  |

Enter position (1-9):

```

RESULT: Thus, the Minimax algorithm was successfully implemented in Python to determine the optimal move for a game.

Experiment 14: Write the python program to implement Apha & Beta pruning algorithm for gaming

Aim

To write and execute a Python program to implement the Alpha-Beta Pruning algorithm for game playing.

Algorithm

1. Start the program.
2. Define the game tree and its leaf node values.
3. Initialize Alpha (α) to negative infinity.
4. Initialize Beta (β) to positive infinity.
5. Define the Alpha-Beta function.
6. If the current node is a terminal node, return its value.
7. For the MAX player, select the maximum value and update Alpha.
8. For the MIN player, select the minimum value and update Beta.
9. If $\text{Beta} \leq \text{Alpha}$, prune the remaining branches.
10. Continue until the optimal value is obtained.
11. Display the optimal value and pruned branches.
12. Stop the program.

Program Code:

```
def alpha_beta (depth, node, alpha, beta, max_player, values):  
    if depth == 2:  
        return values[node]  
    if max_player:  
        best = -999  
        for i in range (2):  
            best = max (best, alpha_beta (depth+1, node*2+i,  
                                           alpha, beta, False, values))  
        alpha = max (alpha, best)  
        if beta <= alpha:
```

```

        break
    return best
else:
    best = 999
    for i in range (2):
        best = min (best, alpha beta (depth+1, node*2+i,
                                alpha, beta, True, values))
        beta = min (beta, best)
        if beta <= alpha:
            break
    return best
values = [3, 5, 2, 9]
result = alpha beta (0, 0, -999, 999, True, values)
print ("Optimal Value:", result)

```

OUTPUT:

```

Branch pruned at depth: 2
Branch pruned at depth: 1

Optimal value: 5

```

RESULT:

Thus, the Alpha-Beta Pruning algorithm was successfully implemented in Python. It reduces unnecessary node evaluation while finding the optimal game move.

Experiment 15: Write the python program to implement Decision Tree

Aim: To write and execute a Python program to implement a Decision Tree algorithm for classification.

Algorithm:

1. Start the program.
2. Create the dataset containing input features and class labels.
3. Calculate the Gini Index for possible splits.
4. Find the split having the lowest Gini Index.
5. Divide the dataset into left and right groups.
6. Create leaf nodes based on the majority class.
7. Construct the Decision Tree.
8. Give test data as input.
9. Use the Decision Tree to predict the class of the test data.
10. Display the predicted class.
11. Stop the program.

Program Code:

```
# Simple Decision Tree
def decision_tree(age):
    if age >= 30:
        return "Eligible"
    else:
        return "Not Eligible"
ages = [20, 25, 35, 40]

for age in ages:
    print ("Age:", age, "->", decision_tree(age))
```

OUTPUT:

```
-----  
Decision Tree Results  
-----
```

```
Input: [27, 40000]
```

```
Predicted Class: 0
```

```
Input: [42, 95000]
```

```
Predicted Class: 1
```

```
Input: [32, 70000]
```

```
Predicted Class: 0
```

RESULT:

Thus, the Decision Tree algorithm was successfully implemented in Python and used to classify the given test data.

Experiment 16: Write the python program to implement Feed forward neural Network

Aim:

To write and execute a Python program to implement a **Feed Forward Neural Network**.

Algorithm

1. Start the program.
2. Define the input data.
3. Define the weights for the neural network.
4. Pass the input values to the hidden layer.
5. Calculate the weighted sum of the inputs.
6. Apply the **Sigmoid activation function**.
7. Pass the hidden-layer output to the output layer.
8. Calculate the output using the corresponding weights.
9. Apply the Sigmoid function to obtain the final output.
10. Display the network output for each input.
11. Stop the program.

Program Code:

```
import math

def sigmoid(x):
    return 1 / (1 + math. exp(-x))

def neural_ network (x1, x2):
    h = sigmoid (x1 + x2)
    output = sigmoid(h)
    return output

inputs = [[0,0], [0,1], [1,0], [1,1]]

for x1, x2 in inputs:
    print ([x1, x2], "->", round (neural_ network (x1, x2), 3))
```

OUTPUT:

```
Feed Forward Neural Network
```

```
-----
```

```
Input: [0, 0]  
Output: 0.6225
```

```
Input: [0, 1]  
Output: 0.6524
```

```
Input: [1, 0]  
Output: 0.6492
```

```
Input: [1, 1]  
Output: 0.675
```

RESULT:

Thus, the Feed Forward Neural Network was successfully implemented in Python, and the output was obtained for the given input data.